

EXACT BILINEAR COMPLEXITY OVER FINITE FIELDS: GUARANTEES FOR A DESCENT HEURISTIC, AND A POOL-FREE SOLVER ROUTE

MOHAMED ALI TEWFIK HAMLIL

ABSTRACT. The rank of a bilinear map is the number of multiplications required to compute it, and deciding it over a finite field is NP-complete. We report on an implementation that attacks the problem from two directions and prove what each one guarantees. For a greedy descent we establish exactness of its first step by Rado–Edmonds, an invariant giving soundness, a termination bound, and a pruning lemma from which the descent’s fixed point is shown to be locally optimal against the *entire* candidate pool rather than against the subset it scanned. We prove that the improvement predicate is invariant under any subgroup of the stabiliser of the current span, which is what makes symmetry reduction sound, and observe that the implementation quotients by a subgroup rather than by the full stabiliser, so that enlarging it is a question of efficiency and never of correctness. Separately we note that the satisfiability route requires no enumeration of rank-one maps at any point, so its space is polynomial in the input where the combinatorial route is exponential: on $\langle 4, 4, 4 \rangle$ the pool is 8.2 TiB and refused, while the formula is 11 MB and written in 1.5 s. A second strand minimises the additions rather than the multiplications, and there the same Rado–Edmonds argument returns the minimum over every change of basis rather than a good answer: on the three operators of a published rank-23 $\langle 3, 3, 3 \rangle$ scheme it takes 221 nonzeros to 128 against 167 for the reference implementation, and we separate the three incomparable cost models in which such a count can be read, since on one operator in nine the fewer nonzeros cost additions. We also record three negative results, since each cost real time to establish.

1. INTRODUCTION

Let $\mathbb{F}$ be a field and let T be a bilinear map, presented as a list of slices $T_1, \dots, T_k$, each an $n \times m$ matrix over $\mathbb{F}$. The *rank* of T , equivalently its *bilinear complexity*, is the least r for which

$$t_{ijl} = \sum_{s < r} a_i^{(s)} b_j^{(s)} c_l^{(s)},$$

that is, the least number of multiplications sufficient to compute T . Strassen’s seven products in place of eight for 2×2 matrix multiplication is the origin of the subject, and determining rank in general is open.

Two facts frame everything below. First, over a finite field the decision problem “is $\text{rank}(T) \leq r$?” is NP-*complete*, not merely NP-hard; the sharper statement is due to Schaefer and Štefankovič, who show that tensor rank over $\mathbb{F}$ is polynomial-time equivalent to the existential theory of $\mathbb{F}$. Membership in NP is not a technicality here: it is what makes a decomposition a certificate of polynomial size, and therefore what makes the problem expressible to a solver at all. Over $\mathbb{R}$ or $\mathbb{Q}$ no such certificate exists, and over $\mathbb{Q}$ the problem is not known to be decidable.

Second, the two natural algorithmic formulations differ in a way that is invisible in their statements and decisive in their space usage. Section 5 makes this precise.

Our contributions are: proofs for a descent heuristic that previously carried measurements and one informal remark (Section 3); the observation that the solver formulation is pool-free and therefore polynomial in space (Section 5); a soundness result for symmetry reduction together with the corollary that closes the gap between it and the implementation (Section 4); a proof that the operator sparsification of the second strand attains the minimum over every change of basis, with the ceiling that makes its scan terminate and the cost models that keep its counts from being read against each other (Section 7); and three negative results (Section 8).

2. PRELIMINARIES

Write $\text{cost}(X) = \sum_i \text{rank}(X_i)$ for a list X of matrices. A rank-one bilinear form is exactly one multiplication, so cost counts multiplications.

Definition 2.1. A list S is *feasible* for T if $\text{span}(S) \supseteq \text{span}(T)$.

Feasibility, rather than equality of spans, is the right notion: S need only *generate* T , so a larger spanning set of smaller total rank is a better answer. Karatsuba’s five products for a four-coefficient product is exactly such a case, and it is why $|S|$ may exceed $|T|$.

We use throughout that $\text{span}(T) \setminus \{0\}$ under linear independence is a vector matroid, and that $w(M) = \text{rank}(M)$ is a non-negative integer weight on it.

3. A DESCENT, AND WHAT IT GUARANTEES

The method has three steps. Step 1 chooses a basis of $\text{span}(T)$ of least total rank. Steps 2 and 3 relax feasibility from “basis” to “spanning set” and descend over a pool G of candidate rank-one maps, adopting any candidate whose addition lowers cost. The two later steps are the same procedure differing only in G : step 2 uses the rank-one terms of T itself, step 3 every rank-one map of the shape, one per scalar class, of which there are $(p^n - 1)(p^m - 1)/(p - 1)^2$.

Theorem 3.1 (Step 1 is exact; [7], Theorem 2.1). *Sorting $\text{span}(T) \setminus \{0\}$ by ascending rank and keeping each element that preserves independence returns a basis of $\text{span}(T)$ of minimum total rank.*

Proof. The procedure is the matroid greedy. By Rado–Edmonds it returns a minimum-weight basis of any matroid under any non-negative weight function. The matroid is the vector matroid of $\text{span}(T)$ and the weight is rank. $\square$

Remark 3.2 (Attribution, and a stronger statement). Theorem 3.1 is not new and is stated here only because everything below depends on it. Nakatsukasa, Soma and Uschmajew pose exactly this minimisation, give exactly this greedy as their Algorithm 1, and make the matroid observation in the same words, calling it a greedy algorithm for an infinite matroid of finite rank [7]. Their Corollary 1 is stronger than the remark below: not only is the optimal *value* independent of tie-breaks, but any other basis of lowest total rank takes the same ranks up to permutation, so the whole multiset is an invariant of $\text{span}(T)$. They also record the caveat that the greedy’s oracle, finding a lowest-rank element outside the span so far, is itself NP-hard, so exactness of the *choice* is not tractability of the *problem*. What is new here is the instantiation of that oracle over $\mathbb{F}(p)$ by enumeration, which their nuclear-norm route has no analogue for.

The *value* is therefore independent of how ties are broken, all minimum-weight bases of a matroid having equal weight; the *basis* need not be. Fixing the enumeration order accordingly buys reproducibility, not optimality.

Theorem 3.3 (Soundness). *Every S the descent holds is feasible for T .*

Proof. By induction. The initial S_0 is a basis of $\text{span}(T)$, so $\text{span}(S_0) = \text{span}(T)$. A step replaces S by a basis of $\text{span}(S \cup \{\varphi\})$, and $\text{span}(S \cup \{\varphi\}) \supseteq \text{span}(S) \supseteq \text{span}(T)$ by hypothesis. $\square$

Consequently a result is never wrong, only possibly large.

Theorem 3.4 (Termination). *The descent performs at most $\text{cost}(S_0) - \text{rank}(T)$ adoptions.*

Proof. A candidate is adopted only when cost strictly decreases. By Theorem 3.3 every intermediate S is feasible, so $\text{cost}(S) \geq \text{rank}(T)$. A strictly decreasing sequence of integers bounded below is finite, of length at most the initial gap. $\square$

The next lemma is what licenses the implementation to discard candidates permanently rather than for a single pass, and it is the hypothesis of Theorem 3.6.

Lemma 3.5 (Pruning is sound). *If $\varphi \in \text{span}(S)$ then φ cannot lower cost, now or at any later stage of the descent.*

Proof. Each S is a minimum-weight basis of its own span, being the output of Theorem 3.1 applied to that span. If $\varphi \in \text{span}(S)$ then $\text{span}(S \cup \{\varphi\}) = \text{span}(S)$, whose minimum-weight basis has weight $\text{cost}(S)$; there is no decrease. By Theorem 3.3 the span only grows, so $\varphi \in \text{span}(S')$ for every later S' and the argument repeats. $\square$

Theorem 3.6 (Local optimality over the whole pool). *At the fixed point, no $\varphi \in G$ lowers cost, including those never tested.*

Proof. Each $\varphi \in G$ was either tested and failed to improve, or discarded, in which case Lemma 3.5 shows it could not have improved. $\square$

Theorem 3.6 is strictly stronger than the loop's stopping condition, which speaks only of the candidates that survived pruning. It is also the claim most exposed to an implementation error, since a defect in the pruning would produce a quietly worse answer rather than a visible failure; it is therefore asserted directly by a test, which rescans the entire pool at the fixed point.

Remark 3.7. No approximation ratio follows. Theorem 3.6 concerns single additions only. A move exchanging two elements at once lies outside the neighbourhood and can pay: $\langle 2, 2, 2 \rangle$ sits at a fixed point of cost 8 while its rank is 7, which is why escaping such plateaus is a separate method rather than a parameter of this one.

Remark 3.8. Step 3 cannot do worse than step 2. Step 2's pool consists of rank-one terms of T , each a scalar multiple of an element of step 3's pool, and φ and $c\varphi$ span the same line. The ordering of the steps therefore costs time and not quality.

4. SYMMETRY REDUCTION

Let $\sigma = (\mu, \nu)$ with μ, ν invertible act on matrices by $M \mapsto \mu^\top M \nu$. This is the rank-preserving action under which a decomposition of T is carried to another decomposition.

Theorem 4.1 (Orbit invariance). *If $\sigma(\text{span } S) = \text{span } S$ then φ lowers $\text{cost}(S)$ if and only if $\sigma(\varphi)$ does.*

Proof. Invertibility of μ, ν gives $\text{rank}(\mu^\top M \nu) = \text{rank}(M)$, so σ preserves weights and $\text{cost}(X) = \text{cost}(\sigma(X))$ for every list X . Being a linear isomorphism fixing $\text{span}(S)$, it carries $\text{span}(S \cup \{\varphi\})$ onto $\text{span}(S \cup \{\sigma\varphi\})$. By Theorem 3.1 minimum weight is a function of the span alone, so the two additions have equal cost. $\square$

Hence one representative per orbit of $\text{Stab}(\text{span } S)$ suffices. Note that the hypothesis concerns the stabiliser of the *current* span and not of T : a quotient computed once and reused as S moves is unsound, not merely weaker.

Corollary 4.2. *The conclusion of Theorem 4.1 holds for every subgroup $H \leq \text{Stab}(\text{span } S)$, with orbits at least as fine.*

Proof. The proof of Theorem 4.1 uses only that each element preserves rank and fixes $\text{span}(S)$, which every element of a subgroup does. Finer orbits yield more representatives, hence a smaller reduction and never a wrong answer. $\square$

Corollary 4.2 is what reconciles the theory with the implementation. On the matrix multiplication path the ambient group is supplied as nine generators, and the code retains those that fix the span; the quotient is therefore by the subgroup they generate and not by the full stabiliser. The corollary shows this is sound, and locates the risk in any future improvement: computing a larger subgroup changes performance and cannot change an answer.

Remark 4.3. The action above is not a symmetry a satisfiability solver can exploit internally. Every established SAT symmetry method acts on permutations of literals commuting with negation, whereas σ mixes coordinates linearly. Only the symmetric group on the r summands is expressible in conjunctive normal form. Group-based reduction must therefore enter from outside the solver, as a partition of the search into independent instances, one per orbit of the first term.

5. TWO FORMULATIONS, AND WHY THEIR SPACE DIFFERS

The combinatorial formulation searches for a subspace of dimension r possessing a basis of rank-one maps, choosing successive basis elements from an explicit pool. The satisfiability formulation introduces one variable per coordinate of each factor and constrains their products to reproduce T .

The two decide the same question and their space requirements are not comparable. The pool has $(p^n - 1)(p^m - 1)/(p - 1)^2$ elements, exponential in the axis lengths; the formula has $r(n_1 + n_2 + n_3) + r n_1 n_2 + r n_1 n_2 n_3$ variables, polynomial in the size of T . The rank-one condition in the second is imposed by the shape of the formula rather than by selection from a list, so no enumeration occurs at any point.

$\langle 4, 4, 4 \rangle$ at $r = 47$	combinatorial	satisfiability
objects held	4,294,836,225	—
space	8.2 TiB (refused)	11 MB
time to produce	—	1.5 s

TABLE 1. The same tensor and the same r , both instruments.

The distinction is not incidental to the implementations. It follows from the formulations: “choose the i -th rank-one map” presupposes an enumeration, while “these coordinates multiply out to T ” does not.

Remark 5.1. Polynomial space is not tractability. For the combinatorial route a lazily generated pool would change space and not time, converting an immediate failure into a computation that does not terminate in practice. For the solver route the distinction is the whole point, since the search itself is delegated.

Remark 5.2. Membership in FNP is realised literally. On a satisfying assignment the implementation reconstructs the rank-one matrices, multiplies them out, and compares against T , rejecting the answer if they differ. A positive answer therefore does not rest on trusting the solver or the encoding. A negative answer may be accompanied by a DRAT refutation checked by an independent program.

6. TWO SLICES: A THEOREM THAT DOES NOT SURVIVE THE FIELD

A tensor with two slices is a matrix pencil $A + xB$, and Kronecker's theory classifies such a pencil up to strict equivalence by its *minimal indices* ε_i, η_j and the *elementary divisors* of its regular part. Ja'Ja' [4], and independently Grigoriev [5], give the rank from that data: over an algebraically closed field,

$$R(T) = \sum_{\varepsilon_i > 0} (\varepsilon_i + 1) + \sum_{\eta_j > 0} (\eta_j + 1) + r + \delta, \quad (1)$$

where r is the size of the regular part and δ counts the elementary divisors of degree at least two. All of the data in (1) is computable over $\text{GF}(p)$ in polynomial time, which is what `pencil_rank/` does: the singular structure from the ranks of one block system, the divisors from a Smith diagonal over $\text{GF}(p)[x]$ taken twice, forwards for the finite divisors and reversed for those at infinity.

Remark 6.1 (The mistake worth recording). Applying (1) with the elementary divisors computed over $\text{GF}(p)$ yields neither side. It is not the rank over the closure, where an irreducible of degree d splits into d distinct linear forms and stops contributing to δ ; and it is not the rank over $\text{GF}(p)$. For (I_4, C) over $\text{GF}(2)$ with C the companion matrix of $x^4 + x + 1$, that reading gives 5. An exhaustive search proves no decomposition with five products exists, walking 1 897 576 nodes to the end of the tree, and exhibits one with six.

Proposition 6.2. Let T be a two-slice tensor over $\text{GF}(q)$. Then $R_{\text{GF}(q)}(T) \geq R_{\overline{\text{GF}(q)}}(T)$, and the right-hand side is given exactly by (1) with the elementary divisors taken over the closure. If moreover T is diagonalisable over $\text{GF}(q)$, meaning it has no minimal indices and every elementary divisor is a linear form of multiplicity one, the inequality is an equality.

Proof. A decomposition over $\text{GF}(q)$ is a decomposition over any extension, so enlarging the field cannot raise the rank; the value over the closure is (1) by [4]. For the second claim, diagonalisability gives P, Q over $\text{GF}(q)$ with PAQ and PBQ both diagonal, and the r matrices $e_i e_i^T$ are rank one, rational over $\text{GF}(q)$, and span a space containing both. So $R \leq r$, and r is the value of (1) here. $\square$

The gap is a field-size effect rather than a defect in (1), and it is not new. Sumi, Miyazaki and Sakata [6] record the condition exactly. Writing k for the number of invariant polynomials of A that do not factor into *distinct* linear factors over K , their Theorem 3.3, after Ja'Ja', gives $\text{rank}_K(E^n; A) \leq n + k$ provided $\text{Card}(K) \geq \deg p_1(A)$; their Theorem 3.5 gives $\text{rank}_K(E^n; A) \geq n + k$ with no hypothesis at all, where Ja'Ja' had the reverse inequality only when $p_1(A)$ splits; and their Proposition 3.4 exhibits (E_3, A) over $\text{GF}(2)$, with A the companion matrix of $x^3 + x + 1$, of rank at least 5, so the cardinality hypothesis cannot be dropped.

That proposition is one of the twelve pencils settled by exhaustion here, and this work measured it at exactly 5 against a count of 4 before finding the statement. So the measurements confirm a theorem rather than exhibiting a gap, and `pencil_rank` computes $n + k$ and tests the cardinality condition: where it holds the rank is settled outright, and where it fails $n + k$ remains a proved lower bound over every field. What is left open is the cost *below* the threshold, which is where the three short rows of that table live and where neither theorem applies.

7. THE OTHER COST: SPARSIFYING THE OPERATORS

A decomposition fixes the multiplications and is silent about the additions, which are set by how many nonzero entries the operators that encode and decode it carry. An operator U may be replaced by $U\Phi$ for any invertible Φ , which computes the same bilinear map in another basis and in general costs fewer additions, so a second minimisation sits beside the first.

Definition 7.1 (Operator sparsification). Given $U \in \mathbb{F}^{N \times d}$ of full column rank, find an invertible $\Phi \in \mathbb{F}^{d \times d}$ minimising $\text{nnz}(U\Phi)$.

The columns of $U\Phi$ are a basis of $\text{col}(U)$, every basis of $\text{col}(U)$ is the column list of $U\Phi$ for exactly one such Φ , and nnz of a matrix is the sum of the Hamming weights of its columns. Definition 7.1 is therefore the request for a basis of $\text{col}(U)$ of least total weight: the problem of Theorem 3.1 with wt where that one has rank, and its solution is the same theorem. The hardness surrounding it is a different literature entirely, and is stated first, because a section proving a minimum without saying what is hard invites the reading that the problem was easy.

7.1. Four names, and two unrelated hardness proofs. The inner question, the lightest nonzero vector of a subspace, is one object the literature names four ways: the *minimum distance* of a linear code over $\mathbb{F}(q)$; the *sparsest vector in a subspace* over $\mathbb{R}$ or $\mathbb{Q}$ [22]; the *spark* of a matrix [20]; and the *girth*, the shortest circuit, of a represented matroid. The identification is stated outright in the dual formulation of [27]: the minimum distance of the code generated by A is the size of the smallest linearly dependent set among the columns of $A^\perp$, which is the shortest circuit of the binary matroid they represent. The greedy’s first call below is exactly this problem and its later calls are the relative version of it, so everything here applies to both.

Over $\mathbb{Q}$ the sparsest-basis problem is NP-hard [18], as is the sparse null space basis problem, and that one even when the optimal value is supplied [19]. It admits no approximation within $2^{\log^{1/2-o(1)} n}$ [10], which is the result the oracles of [11] are built around, and deciding whether a rational matrix has a circuit of size at most k is *strongly* NP-complete [20]. The sharpest statement over $\mathbb{R}$ is that the sparsest vector cannot be approximated within any constant factor [21].

Over a finite field the minimum distance problem is NP-complete over $\mathbb{F}(2)$ [23]; over *every* finite field it is hard to approximate within any constant under randomised reductions, and within $2^{\log^{1-\varepsilon} n}$ under quasi-polynomial ones [26]; and parameterised by the distance bound it is $W[1]$ -hard to approximate within any constant, over $\mathbb{F}(2)$ [27] and over any fixed finite field [28].

Neither version is known to reduce to the other, in either direction. The two proofs are independent, the inapproximability results are twenty-two years apart, and the obstruction has a name: over a finite field one step is free from the second generalised Hamming weight bound, and over $\mathbb{R}$ the same step took a chaining argument and Littlewood–Offord anticoncentration.

Remark 7.2 (Two attributions that are routinely got wrong). [24] proves *coset weight* and *subspace weight* NP-complete, the second being “is there a codeword of weight exactly w ”; its own abstract says the result strongly suggests but does not rigorously imply the minimum distance case, which [23] closed nineteen years later. And [25] proves maximum-likelihood decoding and weight distribution $W[1]$ -hard, and not minimum distance, which it leaves open. Both are commonly given as citations for minimum distance, and were so given here first.

Remark 7.3 (Why the exact method below contradicts none of it). Over $\mathbb{F}(q)$ the implementation lists the q^d elements of the space and takes them in ascending weight. That is exponential in d and not polynomial in the input, so it sits where the hardness does not bite: the finite-field

operators here have d equal to 5 or 6, and q^d is 32 or 64. Nor is the factor an artefact of a naive implementation. Under the strong exponential time hypothesis there is no $q^{(1-\varepsilon)d}$ algorithm for the minimum distance problem over any finite field, for a code of q^d codewords [29], which is a lower bound of exactly this walk's shape. Over $\mathbb{Q}$ the space cannot be listed at all, which is what forces the scan of Theorem 7.6.

7.2. The exact oracle, and why the greedy is minimal.

Theorem 7.4 (Sparsification by the matroid greedy is exact). *Let $\mathcal{C} = \text{col}(U)$ have dimension d . Scanning $\mathcal{C} \setminus \{0\}$ by ascending Hamming weight and keeping each vector that preserves independence returns a basis of $\mathcal{C}$ of minimum total weight, and the Φ carrying the columns of U to that basis attains the minimum of Definition 7.1.*

Proof. The two minimisations are one, the columns of $U\Phi$ ranging over exactly the bases of $\mathcal{C}$ as Φ ranges over the invertible matrices. Linear independence in $\mathcal{C}$ is the vector matroid of any matrix whose columns span it [9, Prop. 1.1.1], and wt is a non-negative integer weight on that matroid. The procedure is the matroid greedy, which by Rado–Edmonds returns a minimum-weight basis of any matroid under any such weight [9, Lem. 1.8.3]. This is Theorem 3.1 again, over a different matroid and under a different weight. $\square$

Remark 7.5 (Attribution). The composition is not new. The driver is [10]’s, reproduced by [11] as its Algorithm 2, and the oracle it calls is their Problem 2.15, the lightest vector of the space lying outside the span of what is already settled; given such a subroutine, they state, Algorithm 2 returns an exact solution. What is supplied here is that oracle, over $\mathbb{F}(q)$ by the enumeration of Remark 7.3 and over $\mathbb{Q}$ by the scan below. Their own two exact oracles, Algorithms 3 and 4, were implemented here as well; they return the same counts on every operator measured, between 88 and 343 times more slowly, so what separates the methods is cost and never the answer. It is worth recording that the same oracle is published a second time under another name: the relative generalised Hamming weight of [30] at $r = 1$ is Problem 2.15 exactly, with a Brouwer–Zimmermann pruning already on it, and neither literature cites the other.

Over $\mathbb{Q}$ the space is infinite and the enumeration is unavailable. A vector of weight w is supported on w of the N coordinates, so ascending support size is ascending weight, and restricted to a support T the vectors of $\mathcal{C}$ solve one small homogeneous system: in reduced row echelon form a vector’s entry on a pivot coordinate is that echelon row’s coefficient, so the unknowns are the echelon rows whose pivot lies inside T , never more than $|T|$ of them, and the equations are the coordinates that are neither pivots nor in T . Walking the supports in ascending size therefore instantiates the greedy. What is not obvious is that the walk ends.

Theorem 7.6 (The greedy never exceeds the Singleton ceiling). *Every vector the greedy of Theorem 7.4 takes has weight at most $N - d + 1$.*

Proof. Fix an information set, that is d coordinates on which $\mathcal{C}$ projects isomorphically. The systematic basis it determines consists of d vectors, each carrying a 1 on its own coordinate of the set, a 0 on the other $d - 1$, and arbitrary entries on the remaining $N - d$; each therefore has weight at most $N - d + 1$. At any point of the greedy fewer than d vectors are held, so their span cannot contain all d systematic vectors and at least one lies outside it. The greedy takes a lightest vector outside the current span, whose weight is consequently at most $N - d + 1$. $\square$

Remark 7.7. Theorem 7.6 is the Singleton bound applied at every step of the greedy rather than only at the first, and it settles a worry rather than improving a bound. Reading an oracle as “search the column subsets of one fixed size” invites the fear that once some vectors are

settled the lightest vector outside their span is heavier than the scan has reached, and is missed. It cannot be. On the three operators of Table 2, where $N = 23$ and $d = 9$, the ceiling is 15 and the largest weights actually taken were 6, 6 and 7.

The scan costs $O\left(\sum_{w \leq W} \binom{N}{w} (w^3 + wN)\right)$ time, with W the largest weight the greedy takes and $W \leq N - d + 1$ by Theorem 7.6, and $\Theta(dN)$ space, the supports being walked rather than listed.

Remark 7.8 (Where the scan stops, and what answers past it). Theorem 7.6 is an upper bound and the scan carries no lower bound, so it halts on having collected a basis rather than on a proof that nothing lighter exists, and its cost is set by the ceiling and not by the answer. On a 16-dimensional subspace of $\mathbb{Q}^{49}$, the left operator of a published rank-49 $\langle 4, 4, 4 \rangle$ scheme, seven of the sixteen vectors need weight 6 or more and the widest level the walk could reach prices at 1.4 PiB, so the implementation prices it in advance and refuses in milliseconds rather than running. Brouwer–Zimmermann [31, 32], coding theory’s standard method for the same subproblem, holds a lower bound from several disjoint information sets alongside the upper one and stops early; it is the first thing to try. Meanwhile a linear programme answers that operator in 0.34 s at 100 nonzeros, which is an upper bound and not the minimum: it would be the minimum if the operator’s column matroid were regular, and it is not, a sixteen-column minor having determinant -2 .

7.3. Against the reference implementation. PLinOpt [17] is the reference implementation in this problem area. It reaches sparsity by a different route, sparse QLUP elimination and a coefficient search bounded to four rows of support with a set of eleven coefficients, and it is a heuristic where Theorem 7.4 is a minimum. Table 2 puts the two on the three operators of a published rank-23 $\langle 3, 3, 3 \rangle$ scheme.

operator	shape	as given	[17]	minimum	seconds
L	23×9	69	59	43	0.338
R	23×9	66	53	42	0.327
P	9×23	86	55	43	0.434
total		221	167	128	

TABLE 2. Nonzeros in the three operators of `3x3x3_23_Grey-221`. The minimum column is the minimum over every invertible Φ , and it is the minimum because Theorem 7.4 says so and not because nothing better was found.

Four things qualify that table, and each is the kind of qualification a count invites.

The two tools are not doing the same job. PLinOpt is downstream of a decomposition: it is handed operators and produces a straight-line program, with in-place accumulation, Tellegen transposition and compaction around it. The work here searches for the decomposition and sparsifies as a second strand, and finds no common subexpressions at all. The table compares one column of that pipeline and not the pipeline.

The middle column is carried and not re-derived here. Those three numbers were measured once, on one machine, with the reference sparsifier built from upstream and given its best settings first: raising its coefficient bound from the default of eleven to twenty and then to forty leaves it where the default does. That binary is not built in the checkout vendored alongside this work, so re-running the column means building the tool first. A carried number reported as a fresh one is the failure this sentence exists to prevent.

P has to be handed over transposed. It is 9×23 and computes 9 outputs from 23 products, so the admissible change of basis acts on its rows and the object is its row space. Given the stored orientation the reference sparsifier reports 9 nonzeros, having recombined the 23 products, which no change of basis on the outputs may do; its own output flags the alternative reading. Transposed it gives the 55 in the table. The mistake would have produced a false defeat rather than a false victory, which is the direction an author is least likely to check.

Sparsity is trivial to improve by returning a different matrix, so each answer was re-checked outside the tool in exact rational arithmetic: rank 9 before, rank 9 after, and rank 9 with the two stacked, so the space returned is the space that was given.

One row of the comparison goes the other way and is the one that matters most here. Over $\mathbb{F}(2)$ and $\mathbb{F}(3)$ the reference sparsifier returns the zero matrix and exits successfully, so on the finite fields where every operator the descent of Section 3 emits lives, there is no column to compare against.

7.4. Fewer nonzeros is not fewer additions. Nonzeros are a proxy for additions and the proxy is imperfect, so a count means nothing until the model it is counted in is named. There are three, and they are not comparable.

how the additions are counted	basis change
(a) $\text{nnz}(U) - \text{rows}(U)$ and so on, no reuse of intermediates	no
(b) the same formula on a sparsified triple, basis maps charged apart	yes
(c) the length of a straight-line program, which may fall below (a)	no

TABLE 3. Strassen’s 18 additions and Winograd’s 15 are model (a); the alternative-basis line [12, 11, 13] is (b), where the basis maps cost $O(n^2 \log n)$ per recursion level and so never reach the leading coefficient; [17] and the $\langle 3, 3, 3 \rangle$ record chain are (c).

This work measures nnz , so it lives in (a) and (b) and structurally cannot see (c). For $\langle 2, 2, 2 \rangle$ that is enough to close the question: fifteen additions are necessary in the standard basis for every rank-7 algorithm, over an arbitrary ring [15, 16]; changing basis is what gets past that bound, and twelve is optimal once it does [12, Thm. 1.2]. The fixtures here reach twelve, $3 + 3 + 6$ against the published $5 + 5 + 8$, which takes Strassen’s leading coefficient from 7 to 5. No search over that fixture can win, and anything that appears to has mismeasured.

Crossing (b) with (c) is what the exact stage is for, its output being available to something that does find common subexpressions. Table 4 does that on three published rank-23 $\langle 3, 3, 3 \rangle$ schemes.

The margin collapses on the two schemes somebody has already worked over, and on one operator of the nine it reverses: `3x3x3_23.58_L` arrives with 49 nonzeros, the exact stage takes it to 43, and the subexpression pass then finds 18 additions where it had found 16 on the original. Six more zeros cost two additions. A basis of least weight is not a basis with the most shared subexpressions, and nothing in Theorem 7.4 says otherwise.

Remark 7.9 (What the right-hand columns are not). Three cautions, without which those figures are not numbers. The subexpression pass is a randomised search taking no seed: four consecutive runs at its default effort, on one unchanged file, gave 22, 22, 22, 21. Every figure in Table 4 is at a loop count of 10 000, five runs a side, best of each, the same effort on both sides, since giving this side more would be the easiest way to fake the comparison. Next, a

scheme	as published		after the exact stage		
	nnz	program	nnz	program	
Grey-221	221	81	128	62	−23%
3x3x3_23_58	175	57	119	56	−2%
3x3x3_23_55	177	56	117	52	−7%

TABLE 4. Model (b) crossed with model (c): the nonzero count, and the length of the straight-line program a subexpression pass finds from it. The last column is the change in that length.

minimum-weight basis is not unique: the count is the minimum and that is proved, but which basis of that weight comes back depends on the order the scan walks its supports, and different ones give the downstream pass different work. An earlier run returning a different 43-nonzero basis of Grey-221’s L reached 17 where this one reaches 18. A figure in those columns is a figure about one basis and not about the method, which promises the nnz column only. Last, two of the nine operators have identical row-weight profiles and identical numbers at every stage; they are almost certainly one operator relabelled, so the nine are eight data points.

Remark 7.10 (What the 62 may not be read against). The right-hand column of Table 4 is (b) crossed with (c), which is a fourth thing again. Three changes of basis, one per operator, go uncharged in it: they are what the sparsification produced, and the alternative-basis accounting charges them separately at $O(n^2 \log n)$ per level. The record for $\langle 3, 3, 3 \rangle$ at rank 23 in model (c) is 55 additions [14], in the standard basis, with nothing to charge. On that same scheme the table says 52, and the arithmetic a reader will do is not available: the 52 leaves three 9×9 basis changes uncharged, the 55 has none, and nothing here computes the leading coefficient of an alternative-basis algorithm with common subexpressions, which is the quantity through which the two would have to meet. Nor do the two records for that shape compete: 61 linear operations in model (b) [11], whose table [13] reproduces, and the 55 in model (c). The 55 implies a leading coefficient near 4.93, below [13]’s lower bound of 5, which is not a contradiction, that bound being stated in model (b) and a program with common subexpressions not being bound by nnz – rows. What the 56 in the same row of Table 4 does say is that the subexpression pass at that effort roughly reproduces its own shipped record, one addition above it, which is the check that the left-hand columns are measured properly at all.

8. NEGATIVE RESULTS

Each of the following cost time to establish and is recorded so that it is not established again.

Fixed- k search with commitment. A finder that asks only questions it expects to be satisfiable was built on an assumed asymmetry between the cost of acceptance and refutation, quoted at two orders of magnitude. Measured with matched flags, the asymmetry is approximately unity; the original figures compared instances encoded with and without symmetry breaking. The method is dominated by the descent on every fixture measured.

Deeper cube splitting. Partitioning a refutation by orbits of the first two terms rather than the first was measured at 2989.8 s against 108.2 s for the undivided instance, with the longest single part exceeding the whole. Even unbounded parallelism does not recover this.

The expensive step of the descent. Step 3 improved the answer on two of four fixtures, by one product each time, at between 58 and 184 times the cost of the first two steps combined. Effort spent accelerating it is spent on the part that mostly does not pay.

9. LIMITATIONS

The descent carries no approximation guarantee, and [8] is why it is unlikely to acquire a good one: approximating the rank of a three-tensor within $1 + 1/1852$ is NP-hard *over any field*, so the obstruction is not an artefact of working over $\mathbb{F}(p)$ and it bounds every method here at once. The exhaustive route is complete but its cost is $\binom{|G|}{r - \dim T}$, which is prohibitive beyond small shapes. The sparsification of Section 7 is exact but priced by Theorem 7.6’s ceiling rather than by its answer, so it does not finish on an operator whose greedy weights are large, and what it minimises is a proxy that Section 7.4 measures going the wrong way on one operator in nine. Reported timings are machine-dependent and are not reproducible in continuous integration; counts are, and are asserted there. Nothing here settles the bilinear rank problem.

REFERENCES

- [1] M. Schaefer and D. Štefankovič, *The complexity of tensor rank*, Theory of Computing Systems, 2018.
- [2] J. Håstad, *Tensor rank is NP-complete*, Journal of Algorithms 11 (1990), 644–654.
- [3] R. Barbulescu, J. Detrey, N. Estibals and P. Zimmermann, *Finding optimal formulae for bilinear maps*, WAIFI 2012.
- [4] J. Ja’Ja’, *Optimal evaluation of pairs of bilinear forms*, SIAM Journal on Computing 8 (1979), 443–462.
- [5] D. Yu. Grigoriev, *Some new bounds on tensor rank*, LOMI preprint, 1978.
- [6] T. Sumi, M. Miyazaki, T. Sakata, *Rank of 3-tensors with 2 slices and Kronecker canonical forms*, Linear Algebra and its Applications 431 (2009), 1858–1868.
- [7] Y. Nakatsukasa, T. Soma, A. Uschmajew, *Finding a low-rank basis in a matrix subspace*, Mathematical Programming 162 (2017), 325–361.
- [8] J. Swernofsky, *Tensor rank is hard to approximate*, APPROX/RANDOM 2018, LIPIcs 116, 26:1–26:9. (Factor $1 + 1/1852$, over any field.)
- [9] J. Oxley, *Matroid Theory*, 2nd ed., Oxford, 2011. (Rado–Edmonds, the matroid greedy.)
- [10] L.-A. Gottlieb and T. Neylon, *Matrix sparsification and the sparse null space problem*, APPROX/RANDOM 2010. (The greedy driver the sparsest-vector oracles are oracles for.)
- [11] G. Beniamini, N. Cheng, O. Holtz, E. Karstadt and O. Schwartz, *Sparsifying the operators of fast matrix multiplication algorithms*, arXiv:2008.03759, 2020.
- [12] E. Karstadt and O. Schwartz, *Matrix multiplication, a little faster*, SPAA 2017; Journal of the ACM 67 (2020), no. 1.
- [13] O. Holtz, J. Hsu, S. Moran, O. Schwartz and N. Wiernik, *Alternative bases for new fast matrix multiplication algorithms*, ACDA 2025.
- [14] S. Karunaratne and A. Idamekorala, *55 additions suffice for 3×3 matrix multiplication at rank 23*, arXiv:2607.28676, 2026.
- [15] R. L. Probert, *On the additive complexity of matrix multiplication*, SIAM Journal on Computing 5 (1976), 187–203.
- [16] N. H. Bshouty, *On the additive complexity of 2×2 matrix multiplication*, Information Processing Letters 56 (1995), 329–335.
- [17] J.-G. Dumas, B. Grenet, C. Pernet and A. Sedoglavic, *PLinOpt: C++ routines for linear, bilinear and trilinear straight-line programs*, github.com/jgdumas/plinopt.
- [18] S. T. McCormick, *A combinatorial approach to some sparse matrix problems*, Ph.D. thesis, Stanford University, 1983.
- [19] T. F. Coleman and A. Pothén, *The null space problem I. Complexity*, SIAM Journal on Algebraic and Discrete Methods 7 (1986), 527–537.
- [20] A. M. Tillmann and M. E. Pfetsch, *The computational complexity of the restricted isometry property, the nullspace property, and related concepts in compressed sensing*, IEEE Transactions on Information Theory 60 (2014), 1248–1259.
- [21] *Inapproximability of finding sparse vectors in codes, subspaces, and lattices*, FOCS 2025; arXiv:2410.02636.
- [22] Q. Qu, Z. Zhu, X. Li, M. C. Tsakiris, J. Wright and R. Vidal, *Finding the sparsest vectors in a subspace: theory, algorithms, and applications*, arXiv:2001.06970, 2020.
- [23] A. Vardy, *The intractability of computing the minimum distance of a code*, IEEE Transactions on Information Theory 43 (1997), 1757–1766.

- [24] E. R. Berlekamp, R. J. McEliece and H. C. A. van Tilborg, *On the inherent intractability of certain coding problems*, IEEE Transactions on Information Theory **24** (1978), 384–386.
- [25] R. G. Downey, M. R. Fellows, A. Vardy and G. Whittle, *The parametrized complexity of some fundamental problems in coding theory*, SIAM Journal on Computing **29** (1999), 545–570.
- [26] I. Dumer, D. Micciancio and M. Sudan, *Hardness of approximating the minimum distance of a linear code*, IEEE Transactions on Information Theory **49** (2003), 22–37.
- [27] A. Bhattacharyya, É. Bonnet, L. Egri, S. Ghoshal, Karthik C. S., B. Lin, P. Manurangsi and D. Marx, *Parameterized intractability of even set and shortest vector problem*, Journal of the ACM **68** (2021), no. 3, article 16.
- [28] H. Bennett, M. Cheraghchi, V. Guruswami and J. Ribeiro, *Parameterized inapproximability of the minimum distance problem over all fields and the shortest vector problem in all ℓ_p norms*, STOC 2023.
- [29] N. Stephens-Davidowitz and V. Vaikuntanathan, *SETH-hardness of coding problems*, FOCS 2019, 287–301.
- [30] R. San-José, *An algorithm for computing generalized Hamming weights and the Sage package GHWs*, ACM Transactions on Mathematical Software **51** (2025), no. 4.
- [31] K.-H. Zimmermann, *Integral Hecke modules, integral generalized Reed–Muller codes, and linear codes*, Technical Report 3-96, Technische Universität Hamburg-Harburg, 1996. (With A. Brouwer’s earlier work, the Brouwer–Zimmermann algorithm.)
- [32] P. Lisoněk and L. Trummer, *Algorithms for the minimum weight of linear codes*, Advances in Mathematics of Communications **10** (2016), 195–207.
- [33] B. D. McKay, *Isomorph-free exhaustive generation*, Journal of Algorithms **26** (1998), 306–324.
- [34] The CASYS team, *Givaro: exact arithmetic over $\mathbb{F}(p)$ and over the rationals*.